

Ex 2.2.10: three reads before the handover re-run

Ex-2.2.9 moved the anchor onto the larger grammar and left three loose ends. Removal was one-sided on the three HSV ops, one seed in twenty fell under the retention gate, and the non-red alignment at op1 sat above the old reference. This notebook reads each off the stored runs, with one small scoring pass for the cube figures and no new training.

The removal miss comes from the rule we used to pick the lines. The projection acts like a change in the hue of red, and the lines that missed are the ones whose answer takes only the saturation or value of the red operand. Red is the one anchored concept, so these reads cannot say whether another concept would come apart the same way.

The retention drop happens before the anneal begins. The op1 alignment rises under either half of the handover, so it belongs to the new grammar rather than to the readout alone. We propose the re-run.

Observations

None of the lines below is a result; ex-2.2.11 will adopt what it needs from here and score it at fresh seeds.

- **Removal:** on the three HSV ops the kept share follows a change of hue on the red operand, and not the to-zero rule we used to pick the removal lines. Where the answer takes only the saturation or value of the red operand (`sat-hsv` and `value-hsv` with red at op2), two thirds of the clean exact match survives the projection. Where the answer takes the hue of red, or the whole color, it goes. One case no counterfactual predicts: `hue-hsv` with red at op1, whose answer needs only the saturation and value of red, and where a third of the clean exact match survives. A linear probe on the stream reads the projected red operand as a hue rotated away from red, at a lower value, and the rotation grows block by block.
- **Retention:** on `handover` the alignment reaches a noisy plateau by epoch 10, and its peak is just the high point of that noise. By the time the anneal begins at epoch 45 the seeds sit at 0.66, a few of them drifting downward. Through the anneal itself every condition is flat, so the gate is reading the noise in the plateau plus the drift. `handover-slot` and `handover-tied` hold their plateaus, so the drift needs the whole-line labeller and the untied readout together.
- **Containment:** $\bar{\alpha}$ at op1 on the non-red lines is 0.28 on `handover`, 0.18 on `handover-slot`, and 0.16 on `handover-tied`, against 0.02 on the control. Each half of the handover raises it, so the untied readout is not the whole story. The  embedding row picks up the axis only under the whole-line labeller: 0.17 on `handover` and zero on `handover-slot`.
- **What we make of it:** score removal on the lines whose answer takes the hue of the red operand, judge retention against the alignment at the start of the anneal (or drop the ratio and report a level), and carry the op1 alignment as a report line rather than a gate.

How to read this

This is a scouting notebook in the shape of [ex-2.2.4](#): no hypotheses, no gates, no verdicts. Everything here comes from the checkpoints, probe set, metrics, and training trajectories that [ex-2.2.9](#) stored. The cube figures also needed the answer distributions and residual states of the red lines, which [ex-2.2.9](#) did not keep, so a scoring-only experiment ([experiment.py](#)) re-ran its projection read on the twenty `handover` checkpoints and stored those.

The reads use the vocabulary of [ex-2.2.9](#), and the report for [ex-2.2.9](#) has the [full glossary](#). A *red line* has a red operand, meaning redness at or above 0.8; that operand sits in slot op1 or op2. The *projection* removes the anchored axis from the residual stream at every slice and position.² *Kept* is the share of the clean expected exact match that survives the projection, over a group of lines. A *removal line* is a red line where setting the R channel of the red operand to zero moves the true answer by at least 0.4 in the unit cube; the removal gate of [ex-2.2.9](#) wanted *kept* to fall under 0.2 on those.

Neither set matches what the training labeller used.¹

Removal on the order-sensitive ops

Ex-2.2.9 scored removal on eleven ops, and the projection cleared the gate on eight. The three that missed are the ones that read operand order: `hue-hsv`, `sat-hsv`, and `value-hsv` each take one HSV attribute from op2 and the other two from op1. On each the miss was one-sided by slot: with red at op1 the projection removed most of the answer on `sat-hsv` and `value-hsv`, and with red at op2 the answer largely survived. `hue-hsv` ran the other way.

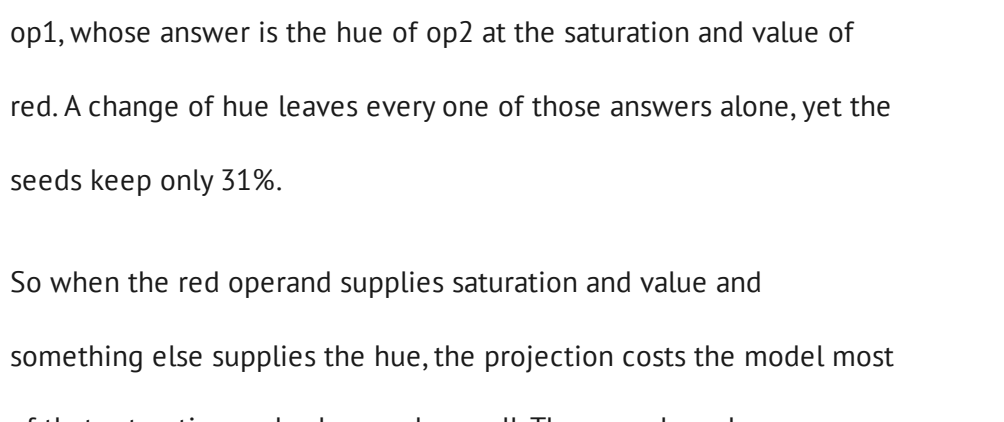
Those removal lines were picked by the *to-zero* rule of ex-2.2.4: a red line counts if setting the R channel of the red operand to zero moves the true answer far. That rule says nothing about which attribute of red the answer actually needs, and it has a blind spot on pure red. Zeroing R on (5, 0, 0) gives black, which has no saturation and no value, so every `sat-hsv` and `value-hsv` line that takes only the saturation or value of red counts as a removal line.

Three counterfactuals

Each is a different reading of what "losing red" should do to the red operand. *To zero* sets its R channel to zero; this is the rule that picked the lines. A *change of hue* replaces the operand by a permutation of its channels, keeping its saturation and value while moving its hue; a line survives when every permutation leaves the true answer within 0.4. A permutation reaches only six hues, a sixth of a turn apart, so this is a coarse hue rotation, and a finer one could behave differently between the hues it samples. *To gray* replaces it by the gray of the same value, which keeps value and removes hue and saturation together.

Op and slot	Red lines	To zero	Change of hue	To gray	Observed, red lines	Observed, removal lines
<code>mix</code> , red at op1	187	0.00	0.00	0.00	0.02 (0.00–0.12)	0.02 (0.00–0.12)
<code>mix</code> , red at op2	178	0.00	0.00	0.00	0.02 (0.00–0.09)	0.02 (0.00–0.09)
<code>hue-hsv</code> , red at op1	406	0.00	1.00	0.00	0.31 (0.11–0.56)	0.31 (0.11–0.56)
<code>hue-hsv</code> , red at op2	404	0.43	0.18	0.90	0.25 (0.08–0.39)	0.17 (0.01–0.32)
<code>sat-hsv</code> , red at op1	406	0.00	0.07	0.80	0.11 (0.05–0.30)	0.11 (0.05–0.30)
<code>sat-hsv</code> , red at op2	404	0.71	1.00	0.04	0.59 (0.30–0.91)	0.65 (0.28–0.97)
<code>value-hsv</code> , red at op1	406	0.04	0.04	0.04	0.07 (0.02–0.15)	0.06 (0.01–0.15)
<code>value-hsv</code> , red at op2	404	0.00	1.00	1.00	0.71 (0.27–0.91)	0.71 (0.27–0.91)

*Predicted and observed kept share, by op and red slot. The three middle columns are the share of red lines whose true answer survives the counterfactual on the red operand (moves by less than 0.4). The last two are what the *handover* seeds kept under the projection, mean with the seed range, on the red lines and on the removal subset. The removal subset is the lines that fail the *to-zero* counterfactual, so the *to-zero* share there is zero by construction.*



*Kept share against the two counterfactuals that could differ from the rule, by op and red slot. Each column is one op and slot. Dots are the kept share of the twenty *handover* seeds on the red lines under the projection, with the seed mean in the larger marker. Beside them: the share of those lines whose true answer would survive a change in the hue of the red operand (plus), or its replacement by gray of the same value (cross). To zero is the rule that picked the removal lines, and it predicts near zero everywhere.*

The projection behaves like the change-of-hue counterfactual, and nothing like the other two. Where a change of hue leaves the answer alone (100% of `sat-hsv` red-at-op2 lines, 100% of `value-hsv` red-at-op2 lines) the seeds keep 59% and 71%. Where it moves the answer (`mix` in either slot, `hue-hsv` with red at op2, `sat-hsv` and `value-hsv` with red at op1), the seeds keep 25% or less.

To gray would have preserved `sat-hsv` red-at-op1 (80%) and `hue-hsv` red-at-op2 (90%). The model keeps neither, so the projection does not turn red into gray.

One slot the hue reading does not predict is `hue-hsv` with red at op1, whose answer is the hue of op2 at the saturation and value of red. A change of hue leaves every one of those answers alone, yet the seeds keep only 31%.

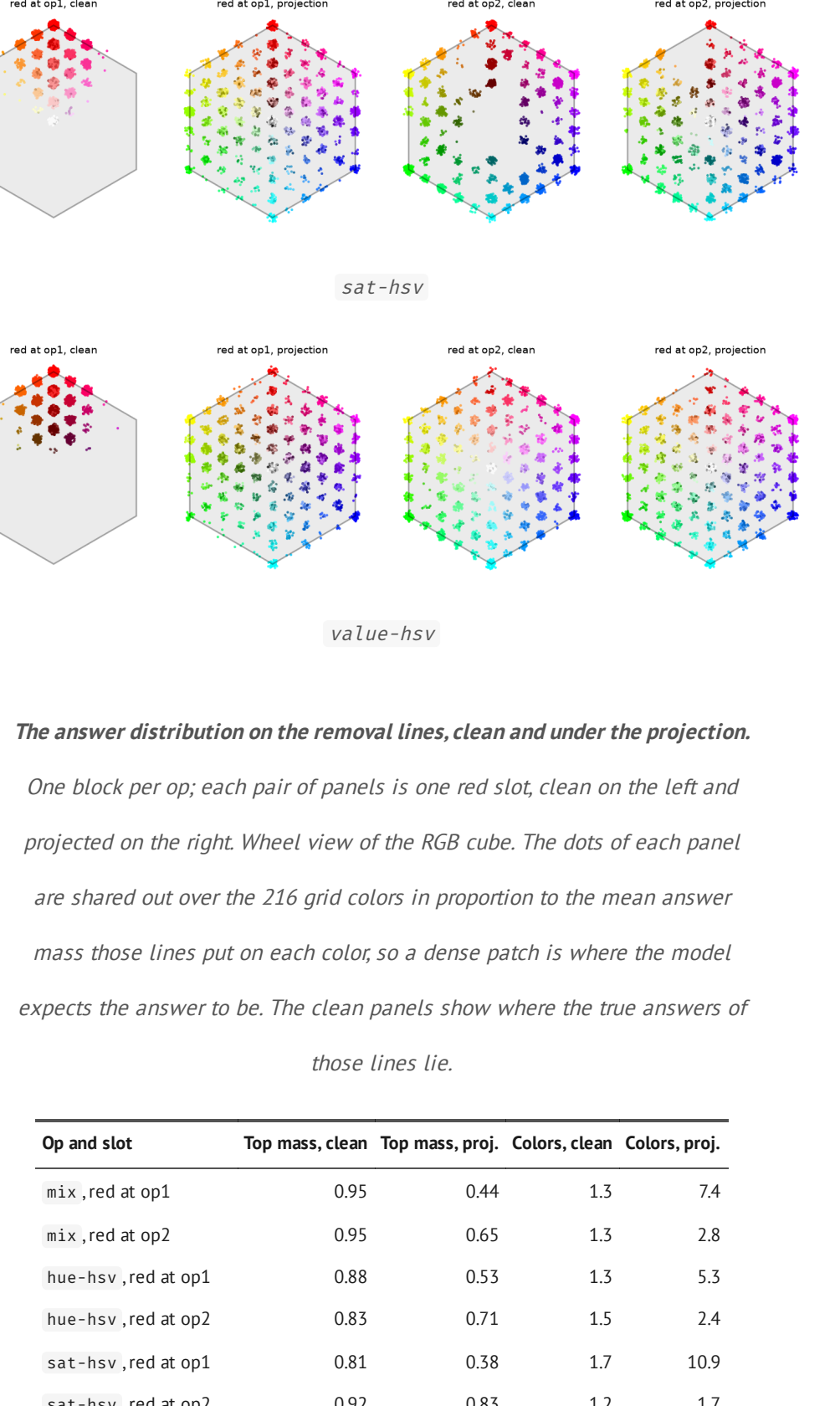
So when the red operand supplies saturation and value and something else supplies the hue, the projection costs the model most of that saturation and value read as well. The same loss shows up more mildly in the two slots that survive: `sat-hsv` and `value-hsv` red-at-op2 keep two thirds where the hue reading says all of it.

One reading is that the removal gate should count the lines whose answer needs the *hue* of the red operand, which is what the *to-zero* rule was meant to proxy. Under a change-of-hue rule, `sat-hsv` and `value-hsv` red-at-op2 drop out of the gate, and `hue-hsv` red-at-op1 drops out with them. What remains are the cases the projection already clears, and the rule rests on the six hues a permutation can reach.

What stays open is whether the partial loss of saturation and value comes from the projection itself or from this checkpoint.

Where the answers go

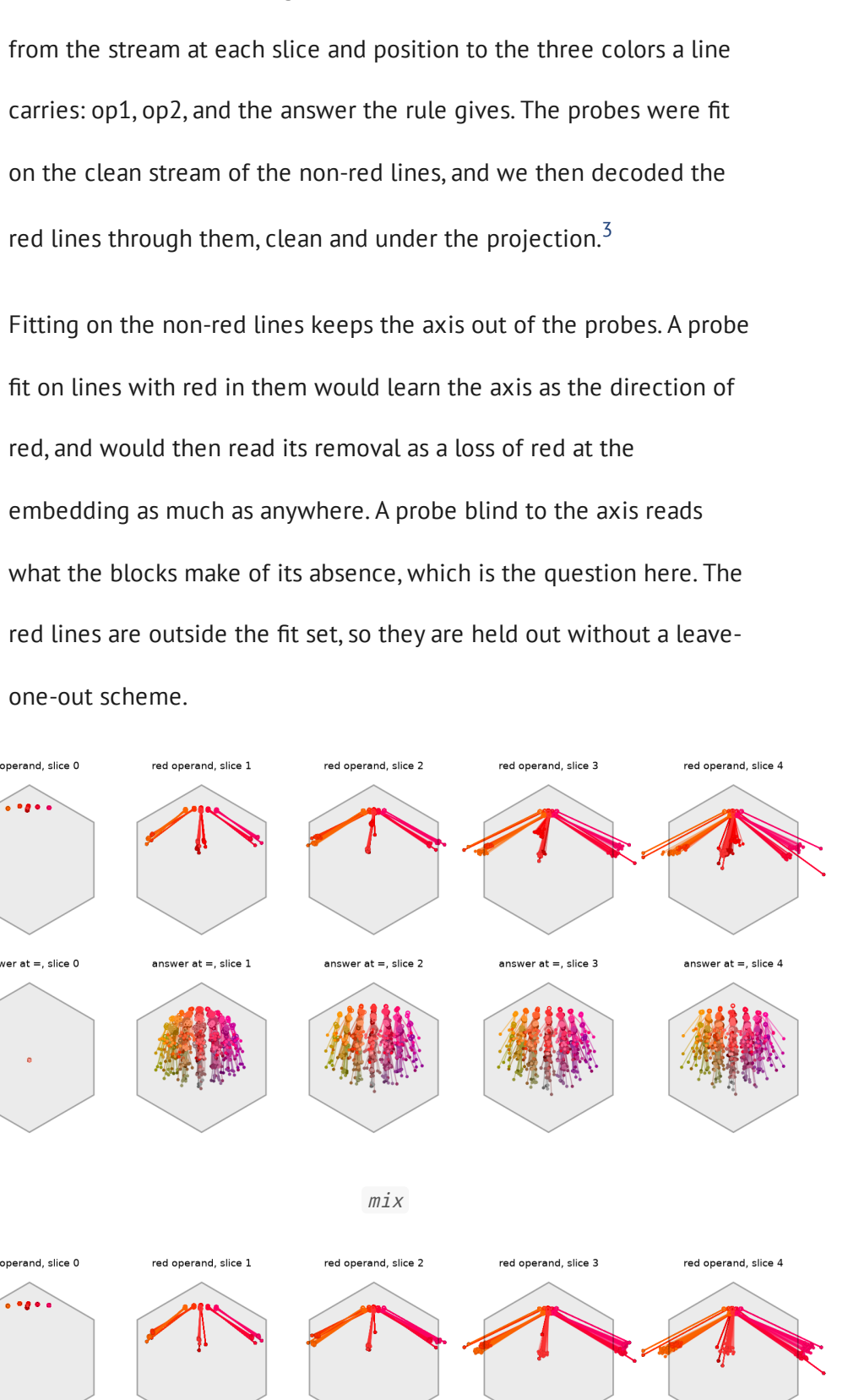
The counterfactual table says which answers are lost. The next three figures show what the model answers instead, on the removal lines of each op and slot, from the twenty *handover* checkpoints. All three use the same view of the RGB cube, the one the probe-cube figures of ex-2.1.1 use: looking down the gray diagonal, so hue runs around the hexagon with red at the top and lightness collapses onto the center.



Greedy answers on the removal lines, clean and under the projection. One block per op; each pair of panels is one red slot, clean on the left and projected on the right, with the number of (line, seed) answers in the title. Wheel view of the RGB cube. Each mark is a greedy answer, placed at its own color and colored by the true answer, with an open ring at the true answer and a stub between them. Marks are sized by how many answers made that move, and a mark sitting on its ring is an answer that matches the truth.

Clean, the greedy answer is the true one on almost every removal line, so the clean panels are where the truth lies. On `mix` the projected answers move toward the center of the cube, and their mean is a gray with a little red left in it. Their hues stay near red: nine in ten sit within a third of a turn of it, which is why the green and blue half of the wheel stays empty. A `mix` answer is the midpoint of its two operands, and a red operand that reads as orange or pink after the projection cannot pull a midpoint to the far side of the wheel.

The greedy answer is only one token. The whole answer distribution says how confidently the model moved, and whether the mass that left the true answer went to one color or spread out. The next figure draws that distribution as a dithered cloud in the cube.



The answer distribution on the removal lines, clean and under the projection. One block per op; each pair of panels is one red slot, clean on the left and projected on the right. Wheel view of the RGB cube. The dots of each panel are shared out over the 216 grid colors in proportion to the mean answer mass those lines put on each color, so a dense patch is where the model expects the answer to be. The clean panels show where the true answers of those lines lie.

Op and slot	Top mass, clean	Top mass, proj.	Colors, clean	Colors, proj.
<code>mix</code> , red at op1	0.95	0.44	1.3	7.4
<code>mix</code> , red at op2	0.95	0.65	1.3	2.8
<code>hue-hsv</code> , red at op1	0.88	0.53	1.3	5.3
<code>hue-hsv</code> , red at op2	0.83	0.71	1.5	2.4
<code>sat-hsv</code> , red at op1	0.81	0.38	1.7	10.9
<code>sat-hsv</code> , red at op2	0.92	0.83	1.2	1.7
<code>value-hsv</code> , red at op1	0.86	0.40	1.4	9.1
<code>value-hsv</code> , red at op2	0.81	0.76	1.6	2.0

How peaked the answer distribution of each line is, clean and under the projection. Mean over the removal lines and the twenty seeds. Top mass is the probability on the most likely grid color. Colors is the effective number of colors, the exponential of the mean entropy: 1 for a certain answer, 216 for a uniform one.

Per line, the projected answer is unsure among a handful of colors rather than spread over the wheel. On the cases the projection breaks, the mass on the top color falls from about 0.9 clean to about 0.5, and the effective number of colors rises from one or two to about 7. On the two cases it leaves alone, the top mass stays at 0.8 or above.

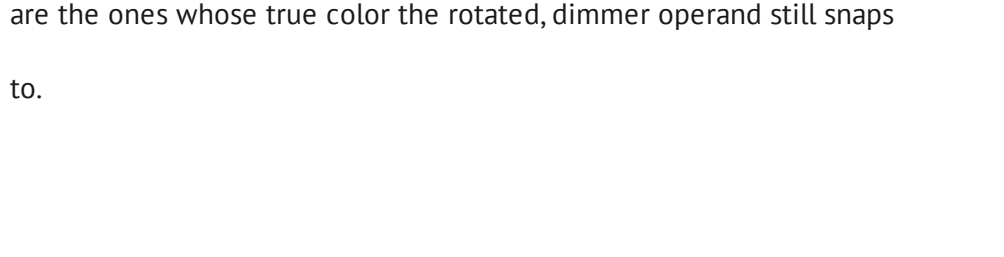
So the wheel-wide spread of the projected clouds is a spread across lines, each moved to its own neighbourhood. That is what we would expect if the operand reads as a hue rotated one way or the other, which is what the probe finds.

The structure inside each cloud is the set of answers the op can produce on the grid, carried around the wheel by the rotation. Take `sat-hsv` with red at op1: the clean answers are reds of every saturation, a ray from white at the center out to red at the top, and projected that ray appears at every hue. With red at op2 the answers are fully saturated colors at the hue and value of op1, which in the wheel view are the rim and the spokes running in toward the center, and the projected panel keeps that skeleton.

What the stream says

The answers show what the model concluded. The residual stream shows what it was working from. For each op we fit a linear probe from the stream at each slice and position to the three colors a line carries: op1, op2, and the answer the rule gives. The probes were fit on the clean stream of the non-red lines, and we then decoded the red lines through them, clean and under the projection.³

Fitting on the non-red lines keeps the axis out of the probes. A probe fit on lines with red in them would learn the axis as the direction of red, and would then read its removal as a loss of red at the embedding as much as anywhere. A probe blind to the axis reads what the blocks make of its absence, which is the question here. The red lines are outside the fit set, so they are held out without a leave-one-out scheme.



Colors of the removal lines decoded from the residual stream, projected against clean. One block per op, mean over the twenty seeds. Top row: the red operand, read at its own position by the probe fit at that slice. One mark per line, at the RGB decoded under the projection and colored by the true color of the operand, with an open ring at the clean decode of the same line and a stub between them, so the stub is what the projection changed. Bottom row: the answer from the rule, read at = and colored by the true answer. Wheel view of the RGB cube. Slice 0 is the embedding, and each later slice is the stream after one more block.

Op and site	slice 0	slice 1	slice 2	slice 3	slice 4
<code>mix</code> , op1@op1	0.95	0.93	0.90	0.88	0.87
<code>mix</code> , op2@op2	0.95	0.92	0.87	0.81	0.78
<code>mix</code> , ans@=	-1.76	0.76	0.92	0.90	0.89
<code>hue-hsv</code> , op1@op1	0.95	0.94	0.91	0.88	0.88
<code>hue-hsv</code> , op2@op2	0.95	0.93	0.88	0.82	0.79
<code>hue-hsv</code> , ans@=	-0.13	0.54	0.85	0.92	0.94
<code>sat-hsv</code> , op1@op1	0.95	0.94	0.91	0.88	0.88
<code>sat-hsv</code> , op2@op2	0.95	0.93	0.88	0.83	0.80
<code>sat-hsv</code> , ans@=	-0.22	0.65	0.88	0.93	0.94
<code>value-hsv</code> , op1@op1	0.95	0.94	0.91	0.88	0.88
<code>value-hsv</code> , op2@op2	0.95	0.93	0.88	0.83	0.79
<code>value-hsv</code> , ans@=	-0.25	0.67	0.91	0.94	0.94

Probe fit on the red lines, R² per slice, mean over seeds. Each probe is a ridge fit on the clean stream of the non-red lines, and is read here on the clean stream of the red lines. op1@op1 reads op1 at position 0, op2@op2 reads op2 at position 2, and ans@= reads the raw answer from the rule at =. The negative fit of the answer probe at slice 0 is expected: at the embedding, = has nothing of the line in it.

Op, site, slice	clean move	clean ΔH	clean S	clean V	proj. move	proj. ΔH	proj. S	proj. V
<code>mix</code> , red operand, slice 1	0.19	0.01	0.73	0.88	0.64	0.10	0.79	0.69
<code>mix</code> , red operand, slice 4	0.27	0.02	0.82	0.88	0.94	0.18	0.82	0.72
<code>mix</code> , answer at =, slice 1	0.08	0.01	0.73	0.73	0.37	0.15	0.49	0.67
<code>mix</code> , answer at =, slice 4	0.07	0.01	0.71	0.73	0.35	0.15	0.51	0.65
<code>hue-hsv</code> , red operand, slice 1	0.18	0.01	0.83	0.89	0.65	0.11	0.79	0.75
<code>hue-hsv</code> , red operand, slice 4	0.26	0.02	0.74	0.88	1.03	0.21	0.87	0.78
<code>hue-hsv</code> , answer at =, slice 1	0.34	0.03	0.70	0.75	0.47	0.07	0.50	0.68
<code>hue-hsv</code> , answer at =, slice 4	0.10	0.01	0.89	0.89	0.36	0.06	0.69	0.77
<code>sat-hsv</code> , red operand, slice 1	0.19	0.01	0.83	0.87	0.66	0.09	0.78	0.61
<code>sat-hsv</code> , red operand, slice 4	0.27	0.01	0.75	0.88	1.05	0.19	0.87	0.61
<code>sat-hsv</code> , answer at =, slice 1	0.26	0.02	0.75	0.86	0.56	0.18	0.39	0.62
<code>sat-hsv</code> , answer at =, slice 4	0.09	0.01	0.79	0.89	0.45	0.14	0.45	0.68
<code>value-hsv</code> , red operand, slice 1	0.19	0.01	0.83	0.88	0.64	0.10	0.80	0.69
<code>value-hsv</code> , red operand, slice 4	0.28	0.02	0.73	0.87	0.94	0.18	0.85	0.72
<code>value-hsv</code> , answer at =, slice 1	0.23	0.02	0.77	0.80	0.51	0.13	0.53	0.72
<code>value-hsv</code> , answer at =, slice 4	0.10	0.01	0.83	0.84	0.38	0.08	0.62	0.76

Where the decoded colors land, clean and under the projection. Mean over the removal lines and the twenty seeds, at the first block and the last. Move is the distance from the decoded color to the true one in the unit RGB cube. |ΔH| is the hue offset from the true color, in turns, so 0.5 is the opposite hue. S and V are the decoded saturation and value; the true operands have both near 1.

At the embedding, the projection changes nothing the probe can see. Clean and projected reads coincide at slice 0, on operand and answer alike, and the small offset both show is the shrinkage of the ridge fit. The probes are blind to the axis, so whatever the projection takes at slice 0 becomes visible only once the blocks have acted on it.⁴

From the first block on, the projected red operand reads as a different color, and the difference grows with depth. The move roughly doubles from slice 1 to slice 4 on every op. In HSV terms it is a rotation of hue away from red, a fifth of a turn by the last slice, at nearly full saturation and with a lower value. In the figure that is the marks sliding along

the upper edges of the hexagon, toward orange on one side and pink on the other, rather than toward the center.

So this is the change-of-hue counterfactual that the kept shares followed, with a loss of value alongside it. M1 saw a related asymmetry when it deleted the hue subspace of an autoencoder (ex-2.7 in ex-preppy, under its ablation figures): red, green, and blue darkened, and yellow, cyan, and magenta lightened. One reading of both is that the primaries sit nearer black on the gray diagonal than the secondaries do, so what remains of red once a hue direction is gone leans toward black.

The slots that take the saturation and value of the red operand have only what the stream keeps of it, so that lost value is the partial loss the counterfactual table could not explain.

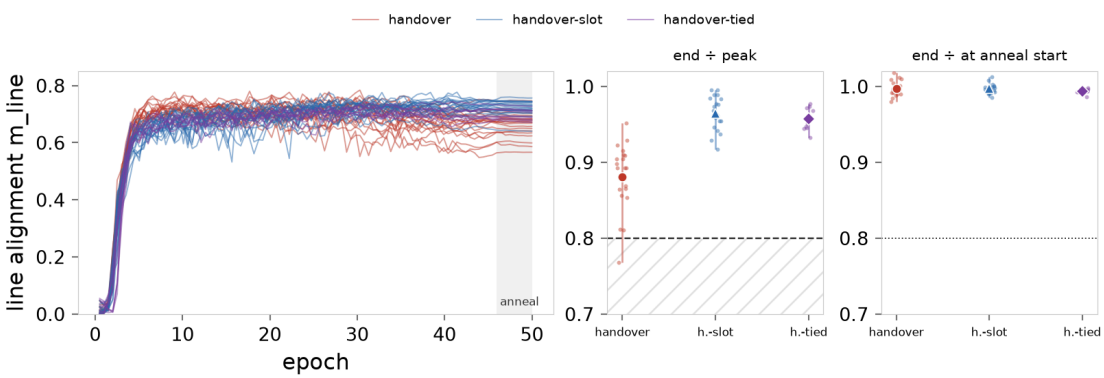
The answer at = follows the operand. Clean, it reaches its ring by the last slice. Projected, it stops short, with a smaller hue offset than the operand and a saturation about two tenths under the clean read. The answers that survive on `sat-hsv` and `value-hsv` with red at op2 are the ones whose true color the rotated, dimmer operand still snaps

to.

Retention and the anneal

The retention read of ex-2.2.9 asks whether the alignment survives the anneal of the anchor weight: every run whose line alignment peaks above 0.2 should end at 0.8 of that peak. One `handover` seed in twenty ended at 0.77, and the seed mean was 0.88 against 0.96 on the references.

The natural reading was that the anneal, over the last tenth of training, lets the alignment slip. The trajectories say otherwise.



Line alignment over training, and two retention ratios. Left: every seed's line alignment against epoch, one thin line per seed, for the three handover conditions; the shaded band is the anneal window, where the anchor weight falls from 0.1 to its floor. Middle: the retention ratio ex-2.2.9 gated, the final alignment over its peak, per seed with the seed mean in the larger marker; the dashed rule is the gate and the hatched side misses it. Right: the final alignment over its value at the start of the anneal, the ratio that would measure the cost of the anneal alone, with the same gate level dotted for reference.

The anneal costs nothing. Over the anneal window the `handover` seeds end at 0.998 of where they started it, and `handover-slot` and `handover-tied` at 0.997 and 0.994. The whole of the drop the gate measured happens earlier.

`handover` climbs to a plateau by epoch 10. Its peak, at 0.75 and epoch 19 on average, is just the high point of a noisy series. We cannot tell whether the high epochs are the ones with more red lines in their batches, since the stored trajectories do not record what each epoch drew; the corpus sampler is seeded, so a replay could tell.

By the start of the anneal the seeds sit at 0.66, a few of them drifting down to the bottom of the band. The two references peak later (epoch 33 on `handover-slot`, 30 on `handover-tied`) because their plateaus are still rising, and they drift less, which is why their ratio comes out higher.

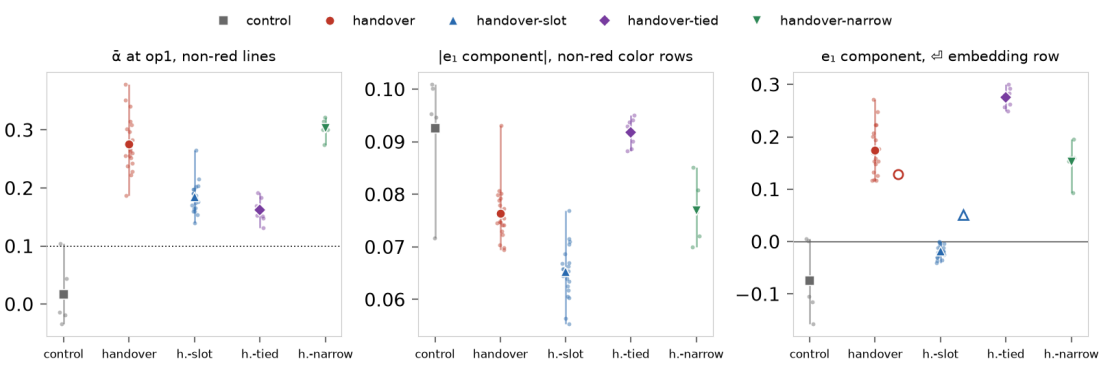
So the ratio of the end to the peak is comparing the last sample of a noisy plateau against the height of that plateau, with a slow drift under a constant anchor weight added on some seeds. That drift needs both the whole-line labeller and the untied readout: either one alone holds its plateau.

Whether the drift matters is a different question from the one the gate asked. The end-of-training alignment is 0.66 on `handover`, against 0.72 and 0.70 on the references. Every read downstream of it (grading, removal, containment) is taken at the end, so that level is what the later experiments inherit. A stepped anneal, which the [backlog item](#) floated, would act on the window where nothing is lost.

Containment under the untied readout

Ex-2.2.9 reported $\bar{\alpha}$, the mean alignment of the non-red lines at op1, at 0.28 on `handover`. The reference from ex-2.2.3 read 0.1, and the old gate was set there. The report put the rise down to the untied readout, which is what the pilot in ex-2.2.7 had suggested.

But `handover` changed two things at once relative to the recipe of ex-2.2.3: the whole-line labeller as well as the readout. Ex-2.2.9 ran one arm with each change undone.



Containment by condition. Per seed, with the seed mean in the larger marker.

Left: the mean alignment of the stream on the non-red lines with the anchor axis at op1, with the reference level from ex-2.2.3 dotted. Middle: the mean absolute axis component of the embedding rows for the 209 non-red color words. Right: the axis component of the $\leftarrow$ embedding row, with the same row of the untied readout as an open marker beside it where the condition has one.

$\bar{\alpha}$ at op1 is 0.28 on `handover`, 0.18 with the slot labeller back, and 0.16 with the readout tied again. Undoing either change takes back about half the rise. So the rise belongs to the handover as a whole, and the untied readout is one of two contributors. `handover-narrow`, which has fewer lines per op, sits at 0.30.

Why either half raises it is open. For the labeller half there is a candidate: under the whole-line labeller a line can earn its label from its answer, and the pull then lands on every position of that line, op1 included. So a non-red operand is pulled toward the axis whenever its answer draws. For the readout half we have no candidate yet. The `backlog` item carries both.

Whatever the mechanism, none of it reaches the embedding table: the non-red color rows hold 0.08 of the axis on `handover` against 0.09 on the control.

The $\leftarrow$ row is a separate matter, and a cleaner one. Its embedding component is 0.17 on `handover` and -0.02 on `handover-slot`, so the whole-line labeller is what puts it there. The pull lands on every position of a red line, $\leftarrow$ included, and the slot labeller never touches that position.

The readout row for $\leftarrow$ is at 0.13 on `handover` too. On `handover-tied` the row reads 0.28, and so does `=`, which is the shared table doing double duty.

The backlog item on the $\leftarrow$ row asks whether the residual redness of the answer position is part of this. Answering that needs a per-position alignment on the red lines, which ex-2.2.9 did not store, so it stays open here.

What we make of it

Three changes to the design of the re-run, and one read to carry.

Removal lines by hue. Define a removal line as a red line whose true answer moves by at least 0.4 under some channel permutation of the red operand. Those are the lines whose answer needs the *hue* of red. On `mix` and the eight other channel-wise ops this is the to-zero set, or close to it. On the three HSV ops it drops the slots that take only saturation or value (`sat-hsv` and `value-hsv` with red at op2), and `hue-hsv` with red at op1 goes with them.

The gate stays at kept under 0.2, and on the lines that remain every `handover` seed already clears it. The permutation rule reaches six hues; if a finer rotation of the operand in HSV, snapped to the grid, ever disagrees with it, the finer one is the rule to keep.

The saturation- and value-taking slots become a second read, with no gate. The kept share there says how much of the saturation and value of the red operand the projection takes with it, which is a cost of the operator worth reporting.

Retention against the start of the anneal. The retention read keeps its name and changes its denominator, to the final alignment over the alignment at the start of the anneal, gated at 0.8 as before. That is the question the read was written to ask.

A high percentile of the plateau, in place of the peak, would soften the ratio and keep the drift inside it; the anneal-start denominator takes the drift out and leaves it to the level line. The early peak and the drift get a line of their own in the report: the level at the end of training, beside the references.

The op1 alignment as a line rather than a gate. $\bar{\alpha}$ at op1 rises under either half of the handover, and the embedding rows do not move, so the old reference of 0.1 belonged to the old grammar. The re-run reports $\bar{\alpha}$ at op1 with `handover-slot` and `handover-tied` as its references, and gates nothing on it. A gate can return once we can name a mechanism.

The  row stays as it is, with its backlog item open. The whole-line labeller is what puts it there, and the slot labeller is not coming back for it. If the edits in M3 ever touch the answer position, this row is the first place to look.

One question this notebook could not settle: whether the partial loss of saturation and value under the projection comes from the operator or from this checkpoint. Answering it would take the same read on `handover-tied` and `handover-slot`, which the scoring pass can run at no design cost, so it is worth adding to the scoring for the re-run.

Method

What ran. A scoring-only experiment over the 20 `handover` checkpoints of ex-2.2.9 (20 scored so far), on the ops `mix`, `hue-hsv`, `sat-hsv`, `value-hsv`, using the probe set of ex-2.2.9. For each checkpoint and op, the task ran two forward passes over every probe line: one clean, and one with the `projection` operator applied at every slice and position. For the red lines it stored the answer distribution over the 216 grid colors, the greedy answer, and the expected exact match under both passes. Its per-group kept shares reproduce those of ex-2.2.9 to the third decimal, which is our check that it ran the same read.

Probes. One ridge regression ($\lambda = 0.01$) per slice, position, and target, from the 64-wide residual state to a color in the unit cube, fit on the clean stream of the non-red lines and read on the red lines. The three targets are op1, op2, and the raw (unrounded) answer the rule gives. The sites read in the figures are each operand at its own position, and the answer at `=` (position 3). We store the decoded coordinates of every red line under both passes for all seeds, and the raw states for the first 5 seeds.

Counterfactuals. For each red line, the red operand is the one with the higher redness. *To zero* sets its R channel to zero. *Change of hue* takes the five other permutations of its channels, and the line survives if all five move the true answer by less than 0.4. *To gray* replaces it by the gray of its value. Answers are the snapped answers of the op, and distances are in the unit cube, the same as `to_zero_move` in ex-2.2.9.

Retention. From the trajectories stored by ex-2.2.9 (100 points over training). The anneal starts at the first point where the anchor weight is under 0.99 of its plateau, and the alignment at that start is the last point before it.

Containment. From the run metrics stored by ex-2.2.9: $\bar{\alpha}$ at op1 as defined there, plus the axis component of each embedding row and, where the readout is untied, each readout row.

-
1. Under the whole-line labeller, each of the three colors in a line draws a label at a small rate that rises steeply with its redness (4% per visit at pure red), and the line is labeled if any of them draws. So a red line is labeled on some of its visits, and a line with a red answer can earn the label too. ↩
 2. The *residual stream* is the running vector the transformer carries from block to block; each block reads it and adds to it. A *slice* is that vector at one depth, and a *position* is one token in the line. ↩
 3. The probe-cube figures of ex-2.1.3 are the precedent. The probes read RGB, so their output lands in the cube with no rotation needed to fit it. A hue probe would be ill-posed, since hue is circular, and saturation and value are piecewise-linear in RGB, so we convert the decoded RGB whenever an HSV number is wanted. ↩
 4. At slice 0 the stream at `=` is the same vector on every line, since nothing of the line has reached that position yet. A probe there can only return one color for every answer: the mean of its fit set, near the center of the wheel. ↩